

Python Coding for AI

Ch1: Python의 직관적 이해

Ch2: Python OOP

Ch3: What is Numpy & Why Numpy?

Ch4: What is Pandas & Why Pandas?

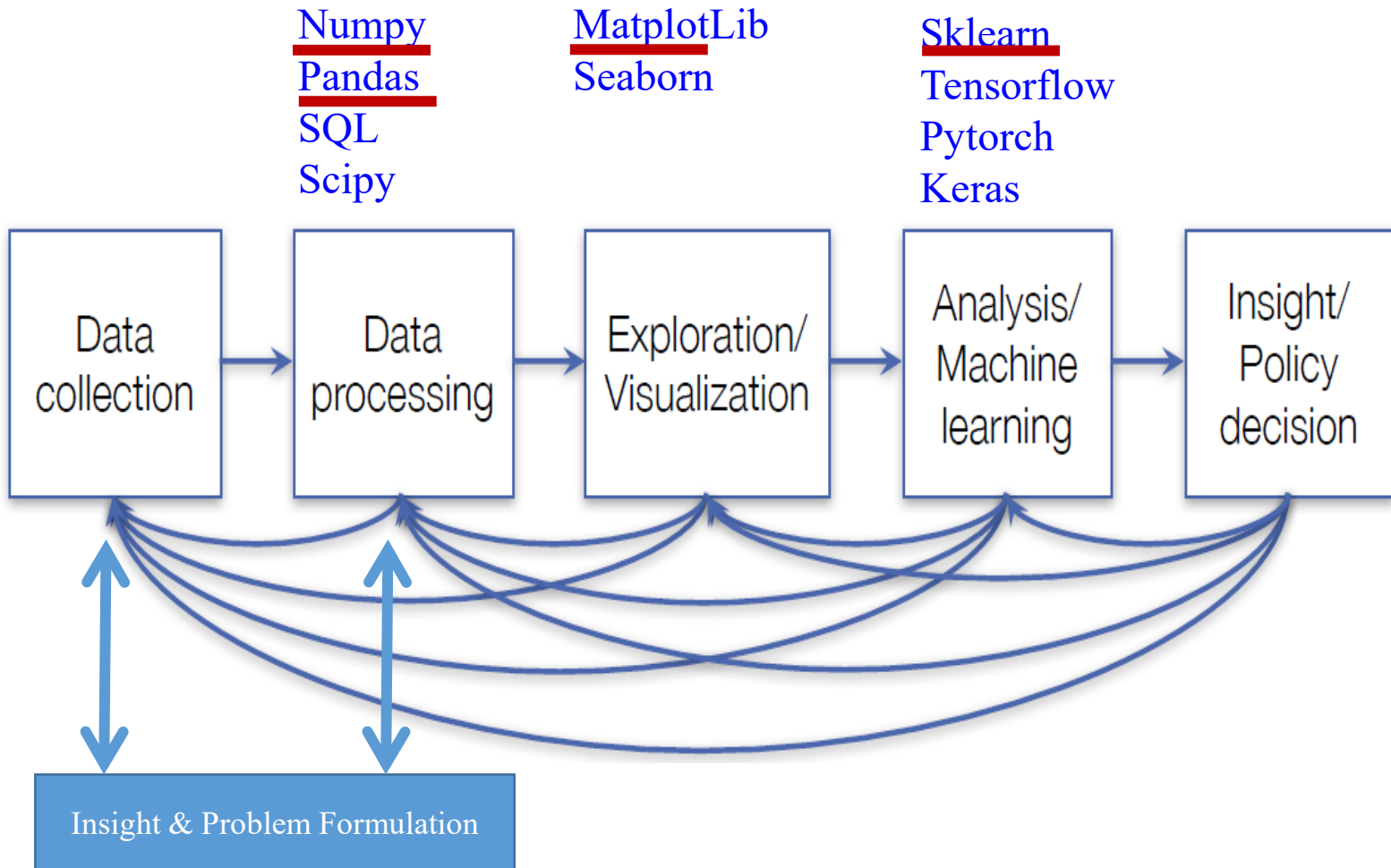
Ch5: What is Matplotlib & Why Matplotlib ?

Ch6: What is Sklearn & Why Sklearn ?

Ch7: What is Data Preprocessing & Why Data Preprocessing ?

서울대 컴퓨터공학부
김형주 명예교수

Major Steps of Data Analytics



Data for Machine Learning and Data Mining

Boston Housing Data

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33

Credit Card Fraud Data

Time	V1	V2	V3	...	V28	Amount	Class
0	-1.35981	-0.07278	2.536347	...	-0.02105	149.62	0
0	1.191857	0.266151	0.16648	...	0.014724	2.69	0
1	-1.35835	-1.34016	1.773209	...	-0.05975	378.66	0
1	-0.96627	-0.18523	1.792993	...	0.061458	123.5	0
2	-1.15823	0.877737	1.548718	...	0.215153	69.99	0
2	-0.42597	0.960523	1.141109	...	0.08108	3.67	0
4	1.229658	0.141004	0.045371	...	0.005168	4.99	0

What Do We Study with Numpy?

	1., 2., 3.	
	4., 5., 6.	

- Creation of Matrix

```
a = np.array([[1, 2, 3], [4, 5, 6]])  
a  
→ array([[ 1.,  2.,  3.],  
         [ 4.,  5.,  6.]])  
a[:, 2]  
→ array([ 3.,  6.] )
```

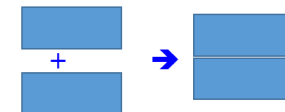
- Accesses to Rows and Columns of Matrix

- Arithmetic and Statistics with Matrix

– Let a, b numpy_ndarray_objects: $a + b$, $a - b$, $a * b$, $a.mean()$,

- Matrix Manipulation

– $reshape()$, $concatenate()$, $vstack()$, $hstack()$, $tile()$,



- Matrix Algebra Operations: Dot, Product, Multiplication, ...

Limitations of Numpy Object [1/2]

Matrix Slicing in Numpy

```
>>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
>>> a[1,:]
array([4., 5., 6.])
```

| 1, 2, 3 |
| 4, 5, 6 |

```
>>> a[:,2]
array([3., 6.])
```

| 1, 2, 3 |
| 4, 5, 6 |

```
>>> a[-1:-2:]
array([[5., 6.]])
```

| 1, 2, 3 |
| 4, 5, 6 |

	AAPL_High	AAPL_Low
Date		
2010-01-04	214.499996	212.380001
2010-01-05	215.589994	213.249994
2010-01-06	215.230000	210.750004
2010-01-07	212.000006	209.050005
2010-01-08	212.000006	209.060005

How about these?

(내용기반 데이터 추출)

2010-01-06 ~ 2010-01-07
사이에 발생한 data 추출?

2010년에 월별로 발생한
data를 Grouping?

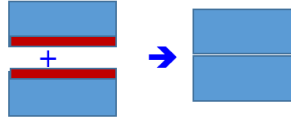


Limitations of Numpy Object [2/2]

- `np.hstack()` : horizontal stacking
 - row숫자가 같은 matrix들에만 적용



- `np.vstack()` : vertical stacking
 - column숫자가 같은 matrix들에만 적용



How about these?
(내용기반 데이터 통합)

	AAPL_High	AAPL_Low
Date		
2010-01-04	214.499996	212.380001
2010-01-05	215.589994	213.249994
2010-01-06	215.230000	210.750004
2010-01-07	212.000006	209.050005
2010-01-08	212.000006	209.060005

	GOOG_High	GOOG_Low
Date		
2010-01-04	629.511067	624.241073
2010-01-05	627.841071	621.541045
2010-01-06	625.861078	606.361042
2010-01-07	610.001045	592.651008
2010-01-08	603.251034	589.110988

	AAPL_High	AAPL_Low	GOOG_High	GOOG_Low
Date				
2010-01-04	214.499996	212.380001	629.511067	624.241073
2010-01-05	215.589994	213.249994	627.841071	621.541045
2010-01-06	215.230000	210.750004	625.861078	606.361042
2010-01-07	212.000006	209.050005	610.001045	592.651008
2010-01-08	212.000006	209.060005	603.251034	589.110988

2 Table의 Join ?

SQL : Query

[1/3]

Data Table들이 DBMS에 있으면 SQL로 해결 가능!, But.....

- To find all instructors in Comp. Sci. dept with salary > 80000

```
select name  
from instructor  
where dept_name = 'Comp. Sci.' and salary > 80000
```

Instructor relation

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000



ID	name	dept_name	salary
83821	Brandt	Comp. Sci.	92000

What if we have numpy 2D Array for Instructors?

```
array([ [ 22222, Einstein, Physics, 95000],  
       [ 12121, Wu,      Finance, 90000],  
       [ 32343, El Said, History, 60000],  
       ....  
       ....  
       ....  
       [ 33456, Gold,    Physics, 87000],  
       [ 76543, Singh,   Finance, 85000] ] )
```

Numpy에서 가능?

SQL : Join

[2/3]

Data Table들이 DBMS에 있으면
SQL로 해결 가능!, But.....

- For all instructors who have taught courses, find their names and the course ID of the courses they taught

```
select  name, course_id
from    instructor, teaches
where   instructor.ID = teaches.ID
```

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

```
select *
from instructor natural join teaches;
```

Instructor ⋈ *teaches*

ID	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

What if we have only numpy 2D Arrays
for Instructors and Teaches ?

Numpy에서 가능?

SQL : Groupby & Aggregation

[3/3]

Data Table들이 DBMS에 있으면
SQL로 해결 가능!, But.....

- Group instructors in each department

```
select *  
from instructor  
group by dept_name;
```

- Find the average salary of instructors in each department

```
select dept_name, avg (salary) as avg_salary  
from instructor  
group by dept_name;
```

What if we have numpy 2D
Array for Instructors?

```
Array([ [ 22222, Einstein, Physics, 95000],  
        [ 12121, Wu, Finance, 90000],  
        [ 32343, El Said, History, 60000],  
        .....  
        .....])
```

Numpy에서 가능?

Instructor relation

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000



ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000



dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Data Collection for Data Analytics

You will typically get data in one of four ways:

1. Directly download a data file (or files) manually
2. Query data from a database
3. Query an API (usually web-based, these days)
4. Scrap data from a webpage



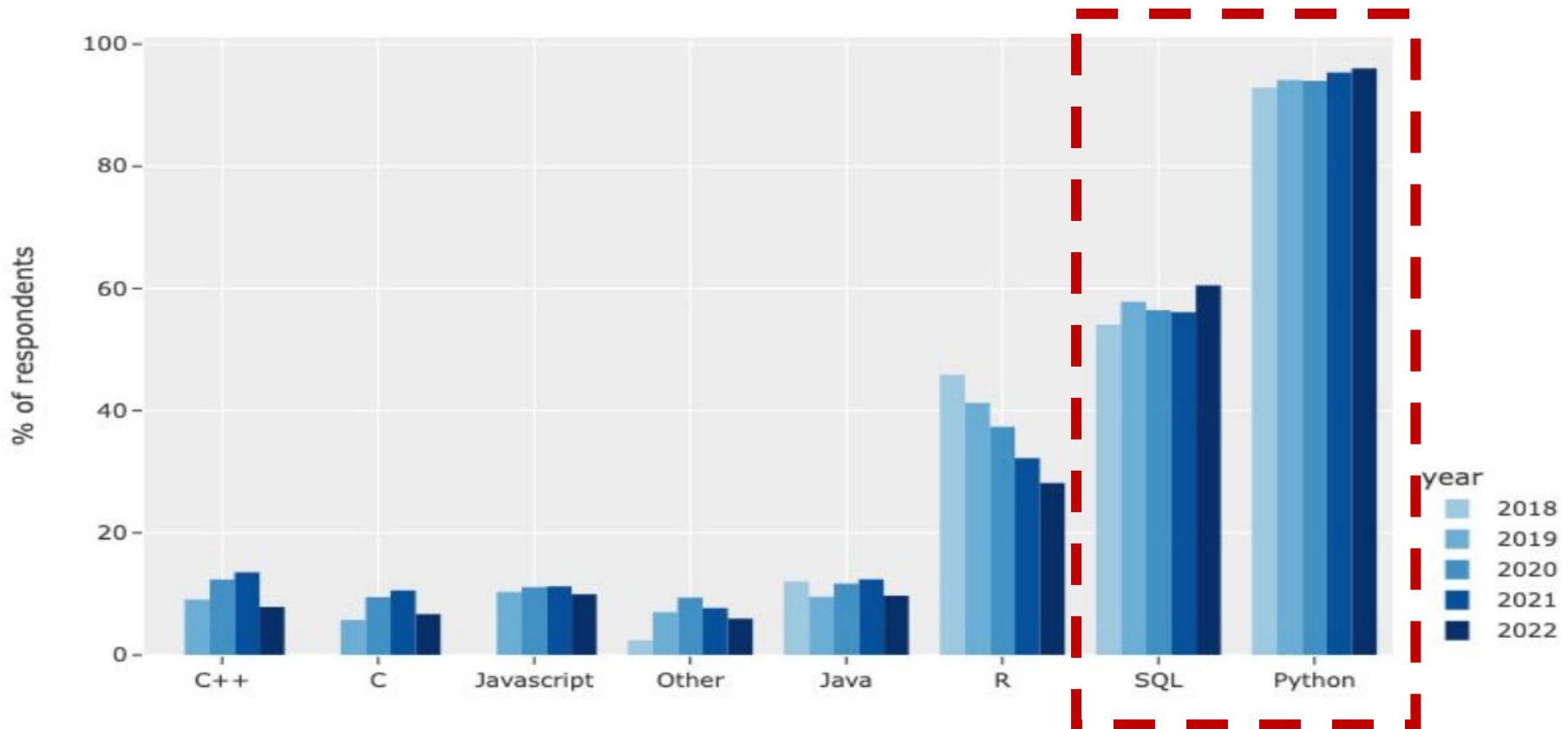
Data Scientist가 Database에서 table data를 추출하여
Python을 가지고 data preprocessing을 하려고 한다면!

Data Format in Data Analytics

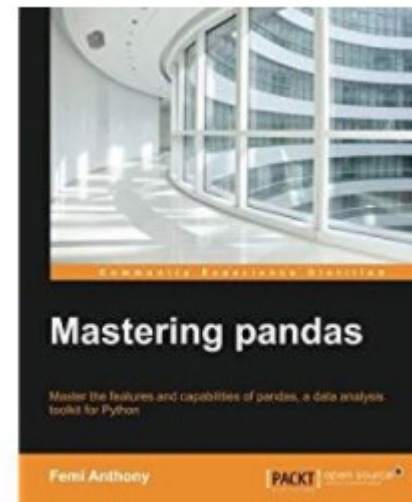
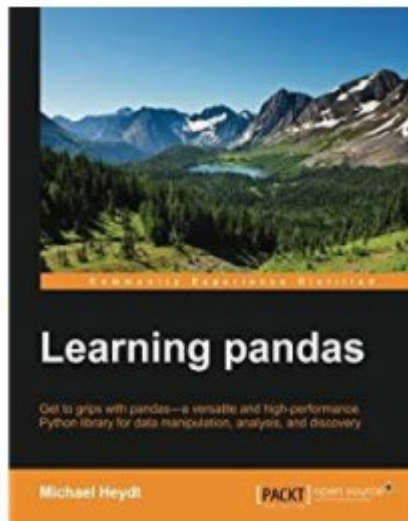
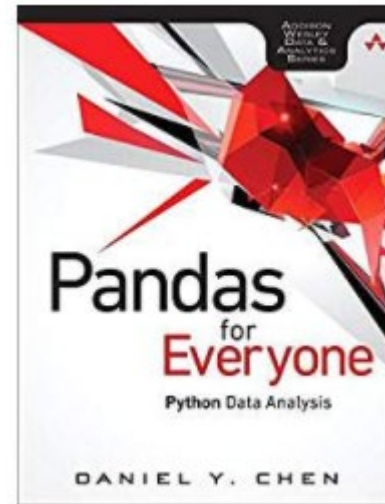
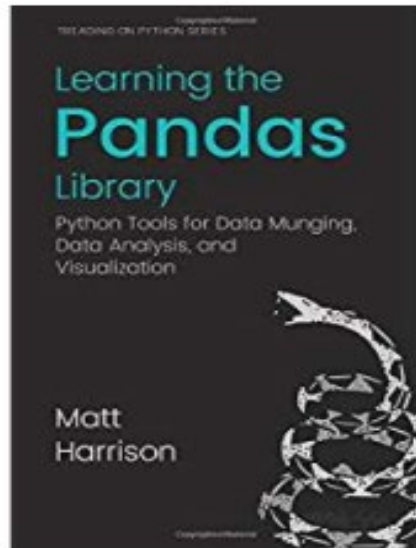
예상되는 2025년 데이터 형식 분포 (추정값 기반)

데이터 형식	대략적 비중 추정	설명
<u>표/관계형/정형 데이터</u>	~40% 이상 (최다 비중)	여전히 핵심이며, 데이터 중심 AI 강조 트렌드 강조
텍스트 및 문서 (비정형)	~30-35% 추정	NLP의 활용 증가, 텍스트 분석 수요 확대
이미지 데이터	~15-20% 추정	컴퓨터 비전 활용 확대
비디오 데이터	~5-10% 추정	실시간 스트리밍, 영상 분석 기술 확대
기타 (로그, 센서, IoT 등)	~10-15% 추정	IoT, 엣지 컴퓨팅, 데이터 통합 수요 영향

Python and SQL remain the two most common programming skills for data scientists



Many PANDAS Books



What is "pandas" Module?

[1/2]

- **Panel data analysis** / **Python data analysis**
- For building and manipulating **relational(tabular) data** both easy and intuitive
- Built on top of **Numpy (2005)**
- **Open source** (<http://pandas.pydata.org>)
 - Original author: **Wes Mckinney** (AQR Capital Management → Lambda Foundary)
 - Now part of the PyData project focused on improving Python data libraries
- **History**
 - Developer **Wes Mckinney** started working on pandas in **2008** while at AQR Capital Management(global investment management firm)
 - Need for **a high performance, flexible analysis tool for financial data**
 - **Wes** was able to convince management to open source library

pandas



Original author(s) Wes McKinney

Developer(s) Community

Initial release 11 January 2008; 15 years ago^[citation needed]

Stable release 2.1.0^[1] / 30 August 2023; 41 days ago

Preview release 2.0rc1 / 15 March 2023

Repository github.com/pandas-dev/pandas 

Written in Python, Cython, C

Operating system Cross-platform

Type Technical computing

License New BSD License

Website pandas.pydata.org 



pandas

pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool,
built on top of the Python programming language.

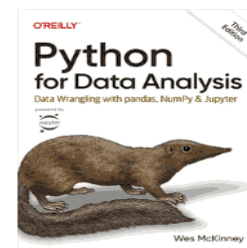
[Install pandas now!](#)

Latest version:
2.1.2

- [What's new in 2.1.2](#)
- Release date:
Oct 26, 2023
- [Documentation \(web\)](#)
- [Download source code](#)

Follow us

Get the book



Getting started

- [Install pandas](#)
- [Getting started](#)

Documentation

- [User guide](#)
- [API reference](#)
- [Contributing to pandas](#)
- [Release notes](#)

Community

- [About pandas](#)
- [Ask a question](#)
- [Ecosystem](#)

With the support of:



Previous versions

- 2.1.1 (Sep 20, 2023)
[changelog](#) | [docs](#) | [code](#)
- 2.1.0 (Aug 30, 2023)
[changelog](#) | [docs](#) | [code](#)
- 2.0.3 (Jun 28, 2023)
[changelog](#) | [docs](#) | [code](#)
- 2.0.2 (May 28, 2023)
[changelog](#) | [docs](#) | [code](#)

[Show more](#)

What is "pandas" Module?

[2/2]

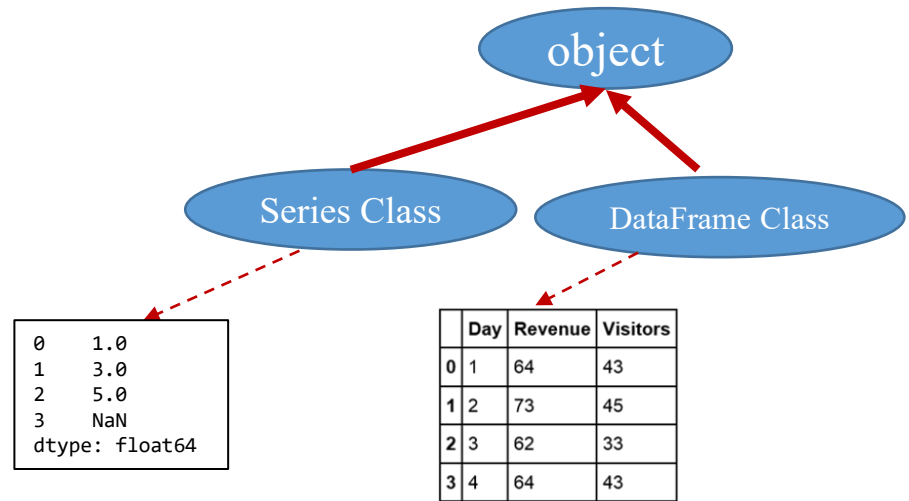
- Primary data structures:

- Series (1-dimensional)**
- DataFrame (2-dimensional)**

- similar to *data.frame* in R

- Things that pandas does well:

- Easy handling of **missing data**
- **Size mutability**: columns can be inserted and deleted: **Add & drop columns**
- **Powerful, flexible group by** functionality: **Groupby & Aggregation**
- **Intelligent label-based slicing, fancy indexing, and subsetting** of large data sets
- **Intuitive merging and joining** data sets: **Join (merge) two data**
- **Robust IO tools** for loading data from flat files (CSV and delimited), Excel files, databases, and web sources



Inside Pandas Module

PANDA Module 속의 Series, DataFrame, DateTimeIndex라는 data type (class)를 OOP로 구현!

Pandas Module

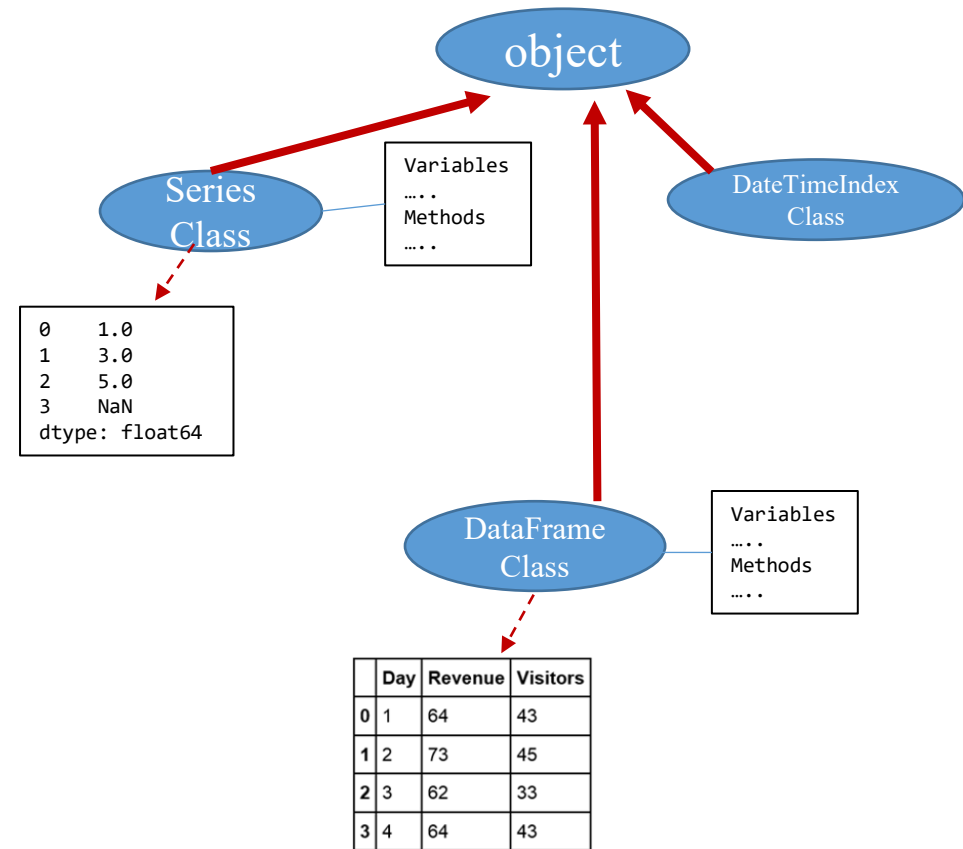
- Class “Series”
 - Variables:
 - Methods: `isnull()`, `notnull()`
- Class “DataFrame”
 - Variables:
 - Methods: `isnull()`, `notnull()`
- Class “DateTimeIndex”
 - Variables:
 - Methods:
- ...

■ Functions

- Functions for Manipulation
 - `isnull()`, `notnull()`, `drop()`, `groupby()`,
- Functions for IO
-

■ Submodules

- `groupby` submodule
- `plotting` submodule
-



Pandas 작업!!!: Series 혹은 DataFrame object를 생성하고, Pandas module에 있는 function을 사용하거나, Series 혹은 DataFrame class속에 있는 method를 사용한다

Creation of Pandas Series Object from Python List

[1/3]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

- 1-D array of Indexed Data
 - Make a pandas.Series object from a python_list

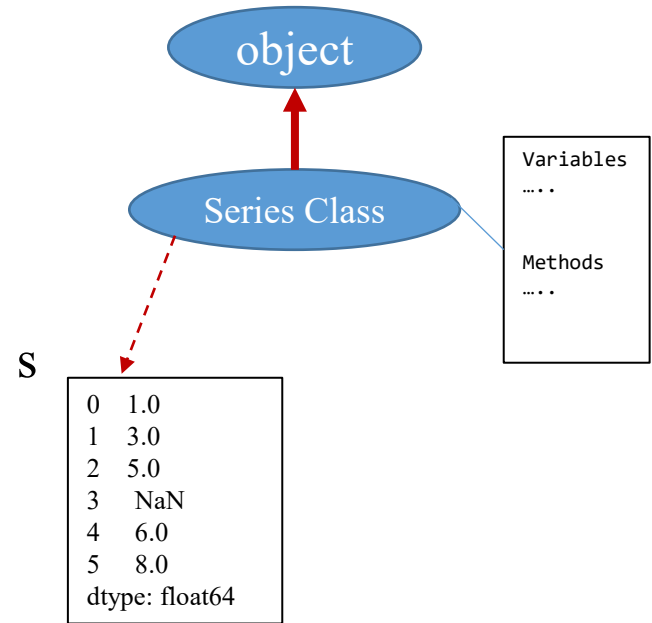
```
s = pd.Series( [1,3,5,np.nan,6,8] )
```

→

0	1.0
1	3.0
2	5.0
3	NaN
4	6.0
5	8.0

dtype: float64

Automatic indexing
index = label = key



** pd.series() 에서 dtype parameter 값을 명시 안하면

- Integer로만 구성되어 있으면 dtype('int64')
- Float number가 1개라도 있으면 dtype('float64')
- np.nan 은 float number



Creation of Pandas Series Object from Python List

[2/3]

- python_list → numpy.ndarray → pandas.Series object

```
np_s = np.array( [1, 3, 5, np.nan, 6, 8] )  
pd_s = pd.Series( np_s )  
  
pd_s
```

→

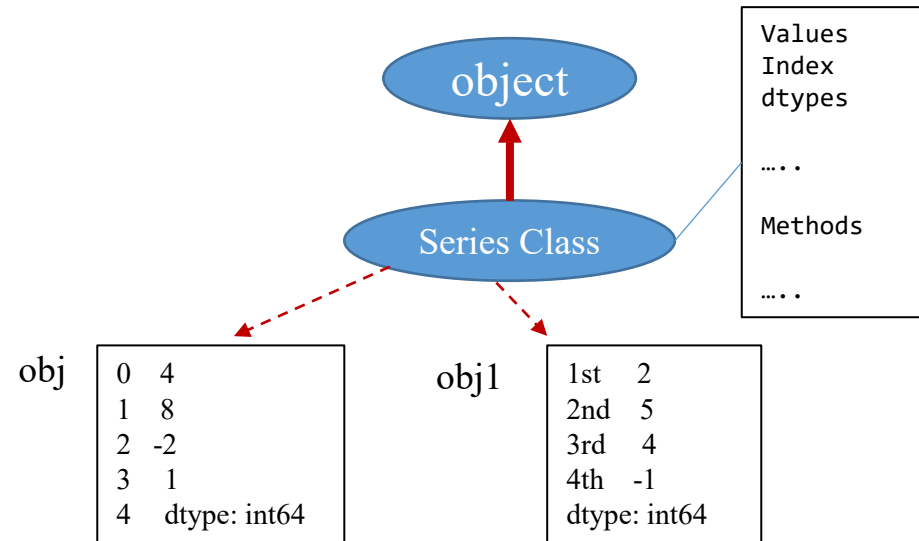
0	1.0
1	3.0
2	5.0
3	NaN
4	6.0
5	8.0

dtype: float64

Creation of Pandas Series Object from Python List [3/3]

```
obj = pd.Series( [4, 8, -2, 1] )
obj
→ 0  4
   1  8
   2 -2
   3  1
dtype: int64

obj.values
→ array( [ 4,  8, -2,  1], dtype=int64)
obj.index
→ RangeIndex( start=0, stop=4, step=1 )
obj.dtypes
→ dtype( 'int64' )
```



```
obj1 = pd.Series( [2, 5, 4, -1], index=['1st', '2nd', '3rd', '4th'] )
obj1
→ 1st  2
   2nd  5
   3rd  4
   4th -1
dtype: int64
```

Table of Contents: Pandas DataFrame Object

- DataFrame Creation and Indexing
- Row/Column Accessing
- Row/Column Manipulation
- Statistics Functions in Pandas
- Groupby and Aggregation

DataFrame Object Creation

- Dataframe is 2-D Array of Indexed Data → Similar to a spreadsheet or SQL table
- Dataframe is the most commonly used pandas object

python dictionary 에서 dataframe object 생성

```
sales_stats = { 'Day':[1,2,3,4,5,6],  
                'Visitors':[43,45,33,43,78,44],  
                'Revenue':[64,73,62,64,53,66] }
```

```
df = pd.DataFrame(sales_stats)
```

→

	Day	Revenue	Visitors
0	1	64	43
1	2	73	45
2	3	62	33
3	4	64	43
4	5	53	78
5	6	66	44

Columns

df

Automatic indexing
(record id/key) = label

pandas-2.py 실행

df

	Day	Revenue	Visitors
0	1	64	43
1	2	73	45
2	3	62	33
3	4	64	43
4	5	53	78
5	6	66	44

object

DataFrame
Class

Variables
....
Methods
....

Automatic Index (자동 인덱스) in DataFrame Object

- **label = key = record id = row id**
- Used for
 - Accessing individual/multiple rows
 - Aligning multiple DataFrames and Series

df: given data

	Day	Revenue	Visitors
0	1	64	43
1	2	73	45
2	3	62	33
3	4	64	43
4	5	53	78
5	6	66	44

Return값이 1개의 Series object
이면 column name, data를 보여
주며, name & dtype도 명시

df.loc[0]

Day 1
Revenue 64
Visitors 43
Name: 0, dtype: int64

Access by Label 값

Return값이 Dataframe object
이면 Table형태로 보여줌

df.loc[:3]

	Day	Revenue	Visitors
0	1	64	43
1	2	73	45
2	3	62	33
3	4	64	43

Access by
Label-range 값

Label

Python Index: 0, 1, 2, 3...

Label이 Python Index순서와 같다면 df.loc[x] 는 df.iloc[x]와 같다

Resetting Index in DataFrame Object

- Using a meaningful column as index

df: given data

	Day	Revenue	Visitors
0	1	64	43
1	2	73	45
2	3	62	33
3	4	64	43
4	5	53	78
5	6	66	44

현재의 Label

Python Index: 0, 1, 2, 3...

```
# Set the "Day" column as an index of the DataFrame.
df = df.set_index('Day')
df
```

	Revenue	Visitors
Day		
1	64	43
2	73	45
3	62	33
4	64	43
5	53	78
6	66	44

현재의 Label

Access by Label 값

df.loc[2]

→ Revenue 73
Visitors 45
Dtype = int64

Access by Label Range

df.loc[1:3]

→ Day Visitors Revenue
1 2 45 73
2 3 33 62
3 4 43 64

Access by Python Index

df.iloc[2]

→ Revenue 62
Visitors 33
Dtype = int64

Access by Python Index Range

df.iloc[1:3]

→ Day Visitors Revenue
1 2 45 73
2 3 33 62

Label이 Python Index순서와 다르다면 df.loc[x] 는 df.iloc[x]와 다르다

Table of Contents: Pandas DataFrame Object

- DataFrame Creation and Indexing
- Row/Column Accessing
- Row/Column Manipulation
- Statistics Functions in Pandas
- Groupby and Aggregation

Accessing Rows by Index [1/3]

Operation	Syntax	Result
Select a column	<code>df[col]</code>	Series object
Select a row by label	<code>f.loc[label]</code>	Series object
Select a row by python index	<code>f.iloc[p-index]</code>	Series object
Slice rows by label range	<code>df.loc[label-range]</code>	DataFrame object
Slice rows by python index range	<code>df.iloc [p-index-range]</code>	DataFrame object
Select rows by boolean expression	<code>df[bool_exp]</code>	DataFrame object

Day를 index로 setting한 상태!

df : given data

	Revenue	Vistors
Day		
1	64	43
2	73	45
3	62	33
4	64	43
5	53	78
6	66	44

`df.loc[ 5 ]`

Revenue 53
Visitors 78
Name: 5, dtype: int64

`df.loc[ 3:5 ]`

	Revenue	Visitors
Day		
3	62	33
4	64	43
5	53	78

Accessing Rows by Index [2/3]

Operation	Syntax	Result
Select a column	df[col]	Series object
Select a row by label	f.loc[label]	Series object
Select a row by python index	f.iloc[p-index]	Series object
Slice rows by label range	df.loc[label-range]	DataFrame object
Slice rows by python index range	df.iloc[p-index-range]	DataFrame object
Select rows by boolean expression	df[bool_exp]	DataFrame object

Day를 index로 setting한 상태!

df: given data

	Revenue	Vistors	
Day			
0	1	64	43
1	2	73	45
2	3	62	33
3	4	64	43
4	5	53	78
5	6	66	44

Label 값

df.loc[1]

Revenue 64
Vistors 43
Name: 1, dtype: int64

Python Index 값

df.iloc[1]

Revenue 73
Vistors 45
Name: 2, dtype: int64

Accessing Rows by Index [3/3]

```
sales_stats = {'Visitors':[43,45,33,43,78,44],  
               'Revenue':[64,73,62,64,53,66]}
```

```
df2 = pd.DataFrame(sales_stats, index=['a','b','c','d','e','f'])  
df2
```

df2

0	a	64	43
1	b	73	45
2	c	62	33
3	d	64	43
4	e	53	78
5	f	66	44

string-based Index!

df2.loc['f']

Label 값

Revenue 66
Visitors 44
Name: f, dtype: int64

df2.loc['c':'e']

Label range 값 ('c', 'e' 포함)

Revenue Visitors
c 62 33
d 64 43
e 53 78

df2.iloc[3:5]

Python Index range 값
(3 ~ 4, 5 미포함)

Revenue Visitors
d 64 43
e 53 78

Accessing the Whole Columns

- RDBMS의 Project Operation과 유사

df2 : given data

	Revenue	Visitors
a	64	43
b	73	45
c	62	33
d	64	43
e	53	78
f	66	44

```
df2[ 'Visitors' ]
```



```
a    43
b    45
c    33
d    43
e    78
f    44
Name: Visitors, dtype: int64
```

df2 table에서 'Visitors' 컬럼
값들을 추출 Series object로
return

```
df2[ ['Visitors'] ]
```



```
Visitors
a         43
b         45
c         33
d         43
e         78
f         44
```

df2 table에서 'Visitors' 컬럼
값들을 추출 DataFrame
object로 return

```
df2[ ['Visitors', 'Revenue'] ]
```



```
Visitors  Revenue
a         43      64
b         45      73
c         33      62
d         43      64
e         78      53
f         44      66
```

df2 table에서 'Visitors' 와 'Revenue'
컬럼들의 값들을 추출 DataFrame
object로 return

Accessing Rows by Attribute (Column) Values

- 내용기반 검색 (= Database Query)
- Content-based Filtering

df2 : given data

	Revenue	Visitors
a	64	43
b	73	45
c	62	33
d	64	43
e	53	78
f	66	44

```
df2[ df2['Revenue'] > 65 ]
```



	Revenue	Visitors
b	73	45
f	66	44

df2 table에서 'Revenue' 값이 65보다 큰 record들을 추출

```
df2[ ( df2['Revenue'] > 65 ) | ( df2['Visitors'] > 70 ) ]
```



	Revenue	Visitors
b	73	45
e	53	78
f	66	44

df2 table에서 'Revenue' 값이 65보다 크거나 혹은 'Visitors' 값이 70보다 큰 record들을 추출

Accessing one Cell by Index Value & Column Name

```
import pandas as pd
# 예제 데이터프레임 생성
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
df = pd.DataFrame(data)
print("Given DataFrame: \n", df)
```

```
print(" Using loc[ ] : ", df.loc[1, 'B'] )
print(" Using loc[ ][ ] : ", df.loc[1]['B'] )
print(" Using at[ ] : ", df.at[1, 'B'] )
print(" Using at[ ][ ] : ", df.at[1]['B'] )
```

Given DataFrame:

df

	A	B	C
0	1	4	7
1	2	5	8
2	3	6	9

Using loc[] : 5

Using loc[][] : 5

Using at[] : 5

TypeError: _get_value() missing

Table of Contents: Pandas DataFrame Object

- DataFrame Creation and Indexing
- Row/Column Accessing
- Row/Column Manipulation
- Statistics Functions in Pandas
- Groupby and Aggregation

Value Update of One Cell in DataFrame Object

```
import pandas as pd
# 예제 데이터프레임 생성
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
df = pd.DataFrame(data)
print("Given DataFrame: \n", df)

# (1, 'B') 위치의 값을 10으로 변경
df.loc[1, 'B'] = 10    # 혹은 df.loc[1]['B'] = 10
                        # 혹은 df.at[1, 'B'] = 10
print("Updated DataFrame: \n", df)
print(df)
```

df

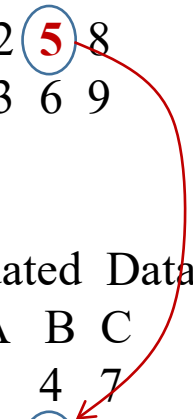
Given DataFrame:

	A	B	C
0	1	4	7
1	2	5	8
2	3	6	9

df

Updated DataFrame:

	A	B	C
0	1	4	7
1	2	10	8
2	3	6	9



Row Manipulation in DataFrame Object

■ Add a new row

df2

Visitors Revenue

a	43	64
b	45	73
c	33	62
d	43	64
e	78	53
f	44	66

df2.loc['g'] = [50, 100]

df2

Visitors Revenue

a	43	64
b	45	73
c	33	62
d	43	64
e	78	53
f	44	66
g	50	100

Add a new row:
Dictionary에
key:value를 add
하는것과 유사

■ Delete an existing row

df2

Visitors Revenue

a	43	64
b	45	73
c	33	62
d	43	64
e	78	53
f	44	66
g	50	100

Drop an
existing row

df2.drop('g', axis=0)

Visitors Revenue

a	43	64
b	45	73
c	33	62
d	43	64
e	78	53
f	44	66

row를 처리하라

Column Manipulation in DataFrame Object [1/3]

■ Original data

df2

	Visitors	Revenue
a	43	64
b	45	73
c	33	62
d	43	64
e	78	53
f	44	66

■ Change the order of Columns

```
pd.DataFrame(df2, columns=['Revenue','Visitors'])
```

	Revenue	Visitors
a	64	43
b	73	45
c	62	33
d	64	43
e	53	78
f	66	44



■ Add a new column: NaN are filled to added column values

```
pd.DataFrame(df2, columns=['Revenue','Visitors', 'Debt'])
```

	Revenue	Visitors	Debt
a	64	43	NaN
b	73	45	NaN
c	62	33	NaN
d	64	43	NaN
e	53	78	NaN
f	66	44	NaN

Add a new
column 'Debt'

Column Manipulation in DataFrame Object [2/3]

■ Original data

df2

Visitors Revenue

a	43	64
b	45	73
c	33	62
d	43	64
e	78	53
f	44	66

■ Add a new column:

df["Debt"] = [np.NaN, np.NaN, np.NaN, 10, 20, 30]

Add a new
column 'Debt'

Revenue Visitors Debt

a	64	43	NaN
b	73	45	NaN
c	62	33	NaN
d	64	43	10
e	53	78	20
f	66	44	30

Column Manipulation in DataFrame Object [3/3]

■ Delete an existing column

```
df3 = pd.DataFrame(df2, columns=['Revenue','Visitors', 'Debt'])
df3
```

→

	Revenue	Visitors	Debt
a	64	43	NaN
b	73	45	NaN
c	62	33	NaN
d	64	43	NaN
e	53	78	NaN
f	66	44	NaN

Delete an existing column 'Debt'

```
del df3['Debt']
```

```
df3
```

Delete an existing column 'Debt'

→

	Revenue	Visitors
a	64	43
b	73	45
c	62	33
d	64	43
e	53	78
f	66	44

```
df3.drop('Debt', axis=1)
```

```
df3
```

column을 처리

→

	Revenue	Visitors
a	64	43
b	73	45
c	62	33
d	64	43
e	53	78
f	66	44

rename() in DataFrame Object

- dictionary, columns 파라미터, index 파라미터를 사용

```
df = pd.DataFrame({'A':[1,4,7], 'B':[2,5,8], 'C':[3,6,9]})
df
```



	A	B	C
0	1	2	3
1	4	5	6
2	7	8	9

```
newcols = {'A': 'col_A', 'B': 'col_B', 'C': 'col_C'}
df.rename( columns = newcols )
```



	col_A	col_B	col_C
0	1	2	3
1	4	5	6
2	7	8	9

- rename(index = newindex_dict) for renaming a row (an index)

```
df.rename( index = {1:'a'} )
```



	col_A	col_B	col_C
0	1	2	3
a	4	5	6
2	7	8	9

Table of Contents: Pandas DataFrame Object

- DataFrame Creation and Indexing
- Row/Column Accessing
- Row/Column Manipulation
- Statistics Functions in Pandas
- Groupby and Aggregation

Common Statistical Functions in Pandas Objects

Series object에도 적용 가능
DataFrame object에도 적용 가능

Method	Description
<code>describe()</code> : descriptive statistics	
<code>count()</code>	Number of non-null observations
<code>sum()</code>	Sum of values
<code>mean()</code>	Mean of values
<code>median()</code>	Arithmetic median of values
<code>min()</code>	Minimum
<code>max()</code>	Maximum
<code>std()</code>	Bessel-corrected sample standard deviation
<code>var()</code>	Unbiased variance
<code>skew()</code>	Sample skewness (3rd moment)
<code>kurt()</code>	Sample kurtosis (4th moment)
<code>quantile()</code>	Sample quantile (value at %)
<code>apply()</code>	Generic apply
<code>cov()</code>	Unbiased covariance (binary)
<code>corr()</code>	Correlation (binary)

이런 기초통계 함수들은 당근으로
알아야 겠죠!

	Revenue	Visitors
a	64	43
b	73	45
c	62	33
d	64	43
e	53	78
f	66	44

df2 : given data

```
df2[ df2['Revenue'] > 65 ].count()
```

```
→ Revenue      2
   Visitors      2
   dtype: int64
```

```
df2['Revenue'].mean()
```

```
→ 63.666666666666664
```

```
df2.cov()
```

```
→
```

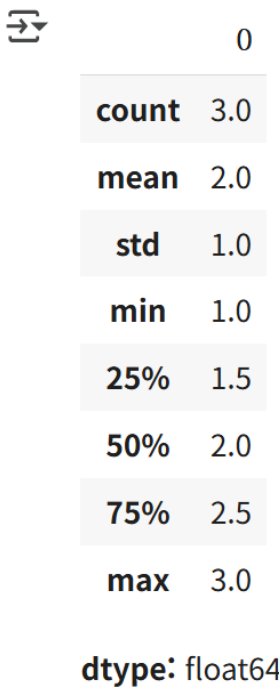
	Revenue	Visitors
Revenue	41.866667	-67.133333
Visitors	-67.133333	239.866667

describe() : Descriptive Statistics of Pandas Objects

- Apply `describe()` to a Series object

Describing a numeric Series object

```
s = pd.Series([1, 2, 3])  
s.describe()
```

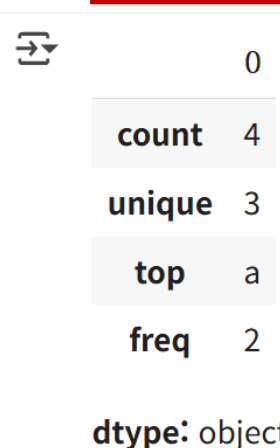


The Jupyter Notebook output for the numeric Series shows a table of descriptive statistics. The index is 0. The statistics are: count (3.0), mean (2.0), std (1.0), min (1.0), 25% (1.5), 50% (2.0), 75% (2.5), and max (3.0). The dtype is float64.

	0
count	3.0
mean	2.0
std	1.0
min	1.0
25%	1.5
50%	2.0
75%	2.5
max	3.0
dtype: float64	

Describing a categorical Series object

```
s = pd.Series(['a', 'a', 'b', 'c'])  
s.describe()
```



The Jupyter Notebook output for the categorical Series shows a table of descriptive statistics. The index is 0. The statistics are: count (4), unique (3), top (a), and freq (2). The dtype is object.

	0
count	4
unique	3
top	a
freq	2
dtype: object	

count() : Count number of non-NaN in Columns and Rows

```
df = pd.DataFrame({ "Person " : ["John", "Myla", "Lewis", "John", "Myla"],  
                    "Age " :    [24., np.nan, 21., 33, 26],  
                    "Single " : [False, True, True, True, False]})
```

df

➔

	Person	Age	Single
0	John	24.0	False
1	Myla	NaN	True
2	Lewis	21.0	True
3	John	33.0	True
4	Myla	26.0	False

각 column 에 있는
값에 대해서

df.count()

➔

Person	5
Age	4
Single	5
dtype:	int64

df.count(axis='columns')

➔

0	3
1	2
2	3
3	3
4	3
dtype:	int64

각 row에 있는
값에 대해서



* Series object를 return

max() : Maximum of the values in the object

```
df = pd.DataFrame({'cate_data':pd.Categorical(['d','e','f']),
'num_data':[1,2,3],'object':['a','b','c']})
df
```

→

	cate_data	num_data	object
0	d	1	a
1	e	2	b
2	f	3	c

df.max()

cate_data f
num_data 3
object c
dtype: object

df.max(axis=0) # 모든 column에 있는 값에 대해서

cate_data f
num_data 3
object c
dtype: object

df.max(axis=1) # 모든 row 에 있는 값에 대해서

0 1
1 2
2 3
dtype: int64

Table of Contents: Pandas DataFrame Object

- DataFrame Creation and Indexing
- Row/Column Accessing
- Row/Column Manipulation
- Statistics Functions in Pandas
- Groupby and Aggregation

Group By in SQL

□ Find the average salary of professors in each department

□ **select** *dept_name*, **avg** (*salary*) **as** *avg_salary*
from *professors*
group by *dept_name*;

Professors

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

**Professors grouped by the
dept_name attribute**

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

**Find the average salary
in each department**

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Groupby and Aggregation in DataFrame Object [1/4]

```
>>> import pandas as pd
```

■ Columns in DataFrame Object

- 2 categorical values : A, B
- 2 numerical values : C, D

`np.random.randn()` :
Return samples from the "standard normal" distribution $N(0, 1)$

```
df = pd.DataFrame({  
    'A': ['foo', 'bar', 'foo', 'bar', 'foo', 'bar', 'foo', 'foo'],  
    'B': ['one', 'one', 'two', 'three', 'two', 'two', 'one', 'three'],  
    'C': np.random.randn(8),  
    'D': np.random.randn(8) })  
df
```

	A	B	C	D
0	foo	one	-1.169421	0.726182
1	bar	one	1.415034	-0.393242
2	foo	two	-0.151896	-0.992431
3	bar	three	-1.528938	-0.202479
4	foo	two	-0.502621	0.691267
5	bar	two	0.247210	1.527112
6	foo	one	0.506265	1.172092
7	foo	three	-0.764634	-0.962124

```
df[ df['A'] == 'foo' ]
```

	A	B	C	D
0	foo	one	-1.169421	0.726182
2	foo	two	-0.151896	-0.992431
4	foo	two	-0.502621	0.691267
6	foo	one	0.506265	1.172092
7	foo	three	-0.764634	-0.962124

```
df[ df['A'] == 'bar' ]
```

	A	B	C	D
1	bar	one	1.415034	-0.393242
3	bar	three	-1.528938	-0.202479
5	bar	two	0.247210	1.527112

Groupby and Aggregation in DataFrame Object [2/4]

- `groupby('column_name')`

```
grouped = df.groupby('A')  
grouped
```

→ `<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000002253C80E3C8>`

```
print(grouped)
```

→ `<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000002253C80E3C8>`

df

	A	B	C	D
0	foo	one	-1.169421	0.726182
1	bar	one	1.415034	-0.393242
2	foo	two	-0.151896	-0.992431
3	bar	three	-1.528938	-0.202479
4	foo	two	-0.502621	0.691267
5	bar	two	0.247210	1.527112
6	foo	one	0.506265	1.172092
7	foo	three	-0.764634	-0.962124

grouped (Not Visible, Not Printable)

Index 역할

A	B	C	D
foo	one	-1.169421	0.726182
	two	-0.151896	-0.992431
	two	-0.502621	0.691267
	one	0.506265	1.172092
	three	-0.764634	-0.962124
bar	one	1.415034	-0.393242
	three	-1.528938	-0.202479
	two	0.247210	1.527112

grouped.get_group('foo')

	A	B	C	D
0	foo	one	-0.093295	-0.507644
2	foo	two	0.366360	-0.000788
4	foo	two	-2.096408	0.338390
6	foo	one	0.695644	2.120278
7	foo	three	-0.143663	0.376266

Groupby and Aggregation in DataFrame Object [3/4]

```
grouped = df.groupby('A')  
grouped (Not Visible, Not Printable)
```

- Aggregation stat functions after grouping



Index로 변환



grouped.mean()

	C	D
A		
bar	0.044435	0.310464
foo	-0.416461	0.126997

grouped.sum()

	C	D
A		
bar	0.133306	0.931391
foo	-2.082306	0.634986

기본적으로
numeric data만
통계값을 출력

Method	Description
count()	Number of non-null observations
sum()	Sum of values
mean()	Mean of values
median()	Arithmetic median of values
min()	Minimum
max()	Maximum
std()	Bessel-corrected sample standard
var()	Unbiased variance
skew()	Sample skewness (3rd moment)
kurt()	Sample kurtosis (4th moment)
quantile()	Sample quantile (value at %)
apply()	Generic apply
cov()	Unbiased covariance (binary)
corr()	Correlation (binary)

Groupby and Aggregation in DataFrame Object [4/5]

```
>>> import pandas as pd
```

- Multiple columns are possible for groupby

```
grouped2 = df.groupby( ['A', 'B'] )
```

grouped2 (Not Visible, Not Printable)

df

	A	B	C	D
0	foo	one	-1.169421	0.726182
1	bar	one	1.415034	-0.393242
2	foo	two	-0.151896	-0.992431
3	bar	three	-1.528938	-0.202479
4	foo	two	-0.502621	0.691267
5	bar	two	0.247210	1.527112
6	foo	one	0.506265	1.172092
7	foo	three	-0.764634	-0.962124

	A	B	C	D
bar		one	1.415034	-0.393242
		three	-1.528938	-0.202479
		two	0.247210	1.527112
foo	one	one	-1.169421	0.726182
			0.506265	1.172092
		two	-0.151896	-0.992431
	two		-0.502621	0.691267
		three	-0.764634	-0.962124

Groupby and Aggregation in DataFrame Object [5/5]

grouped2 (Not Visible, Not Printable)



	A	B	C	D
bar		one	1.415034	-0.393242
		three	-1.528938	-0.202479
		two	0.247210	1.527112
foo		one	-1.169421	0.726182
			0.506265	1.172092
		two	-0.151896	-0.992431
			-0.502621	0.691267
		three	-0.764634	-0.962124



grouped2.mean()



	A	B	C	D
bar	one	1.415034	-0.393242	
	three	-1.528938	-0.202479	
	two	0.247210	1.527112	
foo	one	-0.331578	0.949137	
	three	-0.764634	-0.962124	
	two	-0.327258	-0.150582	

- Get a group's contents

grouped.get_group('bar')



	B	C	D
1	one	1.415034	-0.393242
3	three	-1.528938	-0.202479
5	two	0.247210	1.527112

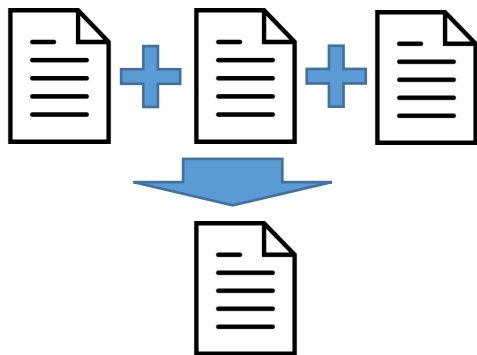
Get contents where the grouping value is 'bar'

Pandas에서 Table 통합

- 여러개의 Pandas DataFrame들의 다양한 통합



- Heterogeneous Table들의 결합에 주로 쓰임
- 예: User 테이블, Purchase 테이블, Basket 테이블
- `pd.merge()` & `df.join()`
- 직관: DB-Join 유사



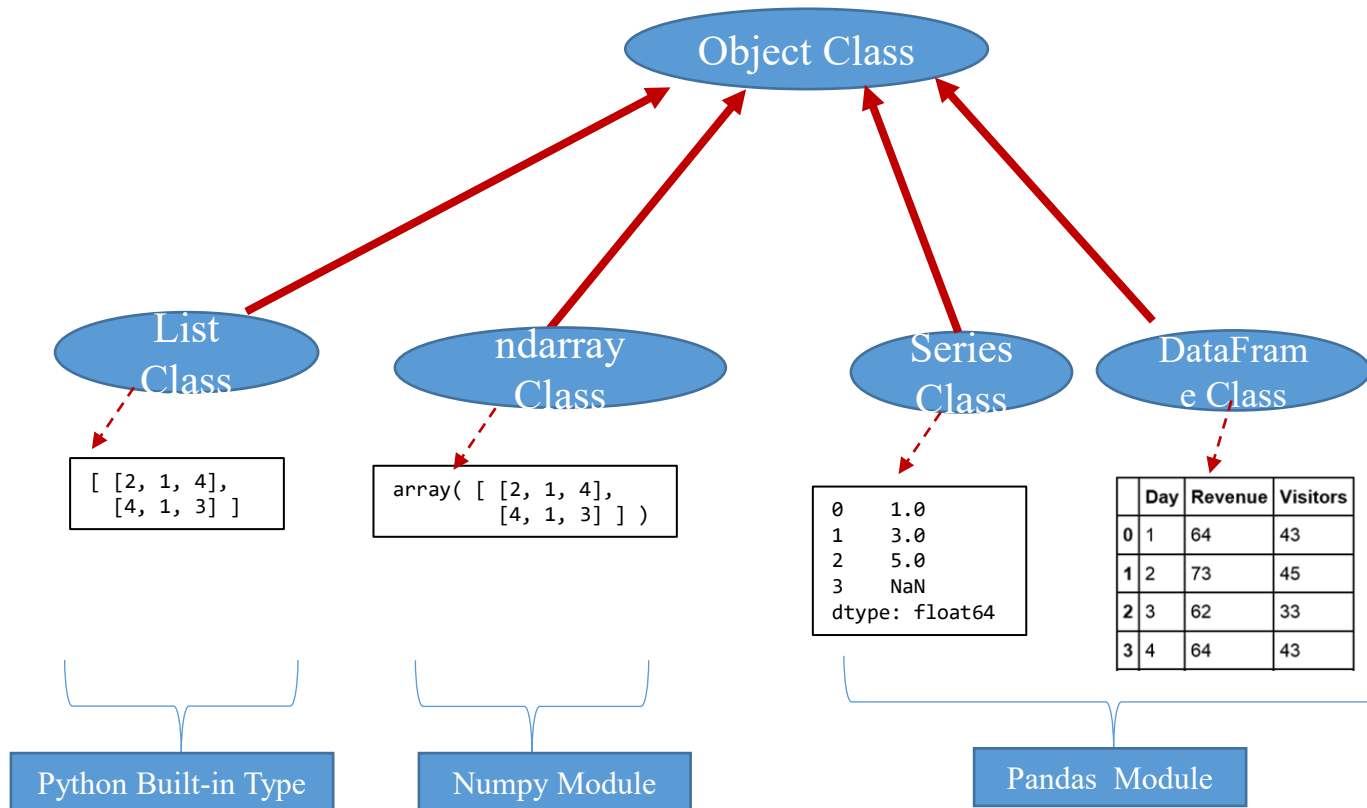
- Homogeneous Table들의 붙임에 주로 쓰임
- 예: xxx_1.csv, xxx_2.csv, xxx_3.csv
- `pd.concat()`
- 직관: DB-Union 유사, Python `append()` 유사

AI Coding을 위해 알아야 하는 Pandas Details

- **Indexing & Selection (loc() vs iloc()):** Pandas 내부 데이터를 접근하기 위해서 이름 기반의 loc()와 정수 인덱스 기반의 iloc()의 차이를 명확히 알고 데이터를 추출
- **Merge & Join (데이터 병합):** 여러 출처의 Pandas 내부 데이터를 on, how 옵션을 사용해 SQL처럼 결합하여 풍부한 데이터셋을 만들
- **Categorical Data (범주형 데이터):** Pandas 데이터에서 문자열 데이터를 category 타입으로 변환하면 메모리 사용량을 대폭 줄이고 연산 속도를 향상
- **Exploratory Data Analysis (EDA 함수):** Pandas 내부의 데이터분포와 특성을 파악하기위해서 다양한 통계함수 (df.info(), df.describe(), df.value_counts(),...)를 구사
- **Groupby & Aggregation (그룹화):** Pandas 내부 데이터를 특정 범주별로 데이터를 묶고 통계량(mean, sum, count)을 산출하는 과정
- **Time Series Handling (시계열 처리):** Pandas 데이터를 to_datetime()으로 변환하여 시계열 데이터를 생성한후 시계열 데이터의 주기를 변경 (resample())하거나 윈도우 연산(rolling())등으로 시간을 기반으로 한 데이터 분석을 수행

Various Array-like Data in Python

다양한
데이터의 모습



- Matplotlib 의 function들에 input으로 가능
- Seaborn 의 function들에 input으로 가능
- SK Learn 의 function들에 input으로 가능